CASO DE NEGOCIO

Se desea construir un módulo básico para gestionar **solicitudes de soporte técnico** dentro de una plataforma interna.

Cada solicitud debe tener:

- Título.
- Descripción.
- Estado: pending, in_progress, resolved.
- prioridad: low, medium, high.
- fecha de creación.
- asignado a (usuario o responsable).

La solución debe permitir:

1. Crear una solicitud.
2. Listar solicitudes.
3. Filtrar por estado.
4. Cambiar el estado de una solicitud.
5. Asignar una solicitud a una persona
6. Visualizar un pequeño dashboard con:
    - Total, de solicitudes.
    - Cantidad por estado.
    - Cantidad por prioridad.

Funcionalidad de Automatizaciones:
Se debe implementar (o al menos diseñar claramente):

Automatización: Envío de correo

Cuando una solicitud es **asignada a una persona**, se debe:

- Disparar una acción de **envío de correo electrónico** al responsable asignado

Consideraciones

- No es obligatorio integrar un proveedor real (como AWS SES, SendGrid, etc.)
- Puede implementarse como:
    - un mock,
    - un log estructurado,
    - o una simulación.
- Lo importante es:
    - cómo se diseña la automatización,
    - dónde se ubica en la arquitectura,

    - cómo se desacopla del flujo principal.
- Se espera que a futuro se puedan incluir más automatizaciones que tengan diferentes triggers y acciones. ¿Cómo abordaría este requerimiento?

PARTE 1: ANÁLISIS Y DISEÑO

Antes de codificar, debes explicar:

- Cómo estructurarías la solución.
- Qué componentes tendría.
- Cómo diseñarías el backend y frontend.
- Qué entidades existirían.
- Cómo modelarías la base de datos.
- Qué endpoints expondrías.
- Cómo organizarías carpetas y módulos.
- Qué patrones o buenas prácticas usarías.
- Cómo implementarías la automatización de envío de correo.

Entregable esperado en vivo

Debes realizar al menos uno de estos artefactos:

- un diagrama simple de arquitectura.
- un diagrama de flujo.
- un modelo entidad-relación simple.
- un esquema en pizarra o en texto estructurado.

Esto permite evaluar tu capacidad de análisis antes de implementar.

PARTE 2: IMPLEMENTACIÓN MINIMA FUNCIONAL

1. Requerimientos

Debes construir una solución funcional que incluya:

Backend

- API REST.
- Endpoints necesarios para.
- Implementación o simulación de envío de correo.
- Validaciones mínimas.
- Estructura clara del proyecto.

Base de datos

- Modelo en PostgreSQL.
- Tabla para solicitudes.
- Campo para asignación.
- Uso adecuado de tipos, restricciones y enums si aplica.

Frontend

- Vista para listar solicitudes.
- Formulario para crear solicitud.
- Filtro por estado.
- Acción para cambiar estado.
- Acción para asignar responsable.
- Sección simple de métricas/resumen.

2. Requerimientos técnicos

Backend

Debe evidenciar:

- Separación en capas.
- Uso correcto de controladores, servicios y acceso a datos.
- DTOs y validación.
- Manejo básico de errores.
- Nombres claros.
- Código legible.
- Diseño adecuado para la automatización (desacoplamiento).

Frontend

Debe evidenciar:

- Componentes organizados.
- Manejo de estado local o global según tu criterio.
- Consumo de API.
- UI simple pero clara.
- Manejo mínimo de loading/error.
- Estructura mantenible.

Base de datos

Debe evidenciar:

- Criterio en modelado.
- Uso correcto de PK

- Timestamps
- Enums o constraints razonables
- Capacidad de pensar en escalabilidad básica

3. Stack recomendado para la prueba

- **Frontend:** React.js
- **Backend:** NestJS
- **Base de datos:** PostgreSQL
- **ORM:** TypeORM
- **UI:** libre (no se evalúa diseño visual avanzado)
- **Infraestructura AWS:**
    - Solo a nivel conceptual (no es obligatorio implementar despliegue)

4. Uso de herramientas de IA

Está permitido el uso de herramientas de IA.

5. Consideraciones importantes

- No se espera una solución completamente terminada.
- Se evaluará principalmente:
    - Cómo abordas el problema.
    - Cómo estructuras la solución.
    - Cómo tomas decisiones.
- Puedes priorizar funcionalidades según el tiempo disponible.
- Es válido dejar partes incompletas si explicas claramente cómo las resolverías.